



Sorokin School

Модуль №2 задание "The Lord of The Locks" [PDF]

Кодовое-название: The Lord of The Locks

Референс: *The Lord of the Rings*

История предисловие к заданию:

[=] Вы несёте **Единый Lock** через земли гонок данных и ловушек дедлоков, чтобы уравновесить хаос в старом легаси-сервисе. На пути вам пригодятся артефакты силы: `synchronized`, `ReentrantLock`, `AtomicLong`, `LongAdder` — каждый со своими чертами и ценой.

Вы соберёте потокобезопасные счётчики, измерите их скорость и устойчивость под нагрузкой, а затем встретитесь с главным врагом — дедлоком — и сломаете его двумя проверенными заклинаниями: тотальным порядком захвата и `tryLock(timeout)` с откатом.

Это путешествие закалит вас: вы выйдете уже как мастер синхронизации, понимающий, где потоки должны ждать, а где им нужно идти.

Цель

1. Понять трейд-оффы между `synchronized`, `ReentrantLock`, `AtomicLong` и `LongAdder` под нагрузкой.
2. Научиться **воспроизводить** и **чинить** дедлок: порядок захвата и `tryLock(timeout) + backoff`.

Замечания и советы по стилю выполнения

- **Java 21 строго.** Все параметры — константами в коде.
- Два задания проекты **независимы**: часть I (Counter Suite) и часть II (Deadlock Simulator) можно запускать и тестировать отдельно.

Задания

Counter Suite — интерфейс и пять реализаций счетчика

Роль: сделать потокобезопасный счётчик пятью способами и **замерить** корректность/скорость под нагрузкой для каждой реализации.

Строгий API

▼ Counter interface

```
1 public interface Counter {
2     void increment();
3     void incrementBy(long delta);
4     long getCount();
5 }
```

Контракт: `increment()` эквивалентен `incrementBy(1)`. `getCount()` — консистентное чтение.

Реализации (каждая — отдельный класс):

- `UnsafeCounter` — без синхронизации (демонстрация гонки).
- `SynchronizedCounter` — критсекция через `synchronized`.
- `AtomicCounter` — на `java.util.concurrent.atomic.AtomicLong`.
- `ReentrantLockCounter` — на `java.util.concurrent.locks.ReentrantLock` (fair = false).
- `LongAdderCounter` — на `java.util.concurrent.atomic.LongAdder`.

Нагрузочный раннер

Класс: `CounterLoad`

Назначение: запустить нагрузку и **самоу напечатать** результат для каждой реализации.

▼

```
1 public final class CounterLoad {
2     public static void runLoad(Counter counter, int threadsCount, int
        iterationsPerThread) {
3         // 1) создать threadsCount потоков
4         // 2) синхронизировать старт (CountDownLatch/CyclicBarrier)
5         // 3) каждый поток делает iterationsPerThread increment()
```

```

6      // 4) засечь время через System.nanoTime()
7      // 5) дождаться завершения всех потоков
8      // 6) посчитать expected = threadsCount * iterationsPerThread
9      // 7) напечатать в одну строку:
10     // "<Impl-X>: expected=<...>, actual=<...>, timeMs=<...>"
11 }
12 }

```

Пример строки вывода (формат):

```
LongAdderCounter: expected=2_000_000, actual=2_000_000, timeMs=112
```

Как запускать и тестировать

Класс: `CountersLoadTesting` → `public static void main(String[] args)`

1. Создайте экземпляры: `UnsafeCounter`, `SynchronizedCounter`, `AtomicCounter`, `ReentrantLockCounter`, `LongAdderCounter`.
2. Константы: `threads = 8`, `itersPerThread = 250_000`.
3. По очереди вызовите `CounterLoad.runLoad(...)` для каждой реализации (каждая распечатает свою строку).
4. Увеличьте нагрузку: `threads = 32`, `itersPerThread = 1_000_000`; протестируйте **только** `AtomicCounter` и `LongAdderCounter` — сравните строки по времени.

✓ Проверь себя

- `UnsafeCounter` показал `actual < expected`.
- У остальных `actual == expected`.
- При 8–32 потоках `LongAdderCounter` по времени **не хуже** `AtomicCounter` (часто — быстрее).

💡 Подсказки

- Чем выше конкуренция, тем заметнее преимущество **LongAdder** за счёт расщепления ячеек.
- `ReentrantLock` требует дисциплины: **всегда** освобождайте в `finally`.
- Для честного сравнения держите одинаковые параметры и порядок прогона.
- Получить имя класса можно при помощи `counter.getClass().getSimpleName();`

Deadlock Simulator — воспроизведение и два исправления внутри одной задачи

Цель: намеренно создать создать дедлок между двумя потоками и показать **два исправления**:

1. **порядок захвата**
2. отпусkanie блокировки: `tryLock(timeout)` + `backoff`.

Детекцию делаем «на глаз»: на «плохой» реализации логи замирают, `join()` не возвращается.

Архитектура и строгий API

- 1) **LockableResource** — общий ресурс (оба типа локов доступны)

Роль: единая модель ресурса, которую можно защищать либо монитором (`synchronized`), либо `ReentrantLock` — для сравнения подходов.

```
1 public final class LockableResource {
2     private final String name;        // Например: "Name-A" / "Name-B"
3     private final int id;             // Для глобального порядка (фикс №1)
4     private final ReentrantLock lock = new ReentrantLock();
5     private final Object monitor = new Object(); // Для synchronized-блоков
6
7     // constructor, getters
8 }
```

Логирование: в местах захвата/освобождения обязательно печатайте имя потока и ресурс — см. `Logger` ниже.

- 2) **LockableResourceProcessor** — стратегий три

```
1 public interface LockableResourceProcessor {
2     /**
3      * Выполняет критическую секцию над двумя ресурсами r1 и r2.
4      * Внутри реализуется выбранная стратегия захвата (зависит от реализации).
5      */
6     void processResources(LockableResource r1, LockableResource r2) throws
7         InterruptedException;
8 }
```

- A. **WrongLockableResourceProcessor** — воспроизведение дедлока

- **Идея:** Поток-1 берёт `r1`, затем (через короткую паузу) пытается взять `r2`. Поток-2 — наоборот (`r2` → `r1`).
- **Чем захватываем:** `synchronized (r.monitor())` (проще увидеть зависание), допускается сделать вариант на `lock.lock()`.
- **Тонкости реализации:**
 - Между захватами вставьте `Thread.sleep(100..200)`, чтобы увеличить шанс взаимной блокировки.
 - Все входы/выходы и попытки — логировать: «взял r1», «пытаюсь r2», «внутри критсекции», «выход».
- **Ожидаемо:** на некоторых итерациях оба потока зависают; `join()` не возвращается.

B. `OrderedLockableResourceProcessor` — фикс №1: тотальный порядок

- **Идея:** всегда захватывать ресурсы в одном порядке (по `id`).
- **Чем захватываем:** также через `synchronized (r.monitor())` (для наглядного сопоставления с Wrong).
- **Поведение:** перед работой отсортируйте `(r1, r2)` по `id` и захватывайте **строго** в этом порядке.
- **Ожидаемо:** дедлок **исключён**, все итерации успешно завершаются.

C. `TryLockLockableResourceProcessor` — фикс №2: tryLock + backoff

- **Идея:** вместо безусловного второго захвата используем `reentrantLock().tryLock(timeout)`. Если не взяли — **освободить первый**, сделать короткий **backoff (10–30 мс)** и повторить.
- **Чем захватываем:** только `ReentrantLock`, т.к. у монитора нет `tryLock`.
- **Ожидаемо:** дедлок **исключён**; в логах видны ретрай/откаты и успешные завершения.

▶ Как запускать и тестировать

Класс: `DeadlockSimulatorDemo` → `public static void main(String[] args)`

Константы в коде:

```
1 final int iterations = 1000;
2 final long pauseBetweenLocksMs = 100;    // для Wrong
3 final long tryLockTimeoutMs = 200;       // для TryLock
```

```
4 final long backoffMs = 20; // для TryLock
```

Важно: блоки A/B/C запускайте **по отдельности** (или тремя `main`-классами), чтобы «плохой» вариант не повесил весь процесс.

A) Wrong (демо дедлока)

1. Создайте `LockableResource rA = new LockableResource(1, "A");` и `rB = new LockableResource(2, "B");`.
2. В цикле `for (i = 1; i <= iterations; i++)`:
 - Создайте процессор: `new WrongLockableResourceProcessor(/*исп. synchronized*/)`
 - Поднимите 2 потока:
 - T1: `processResources(rA, rB)`
 - T2: `processResources(rB, rA)`
 - `t1.start(); t2.start(); t1.join(); t2.join();`
3. **Ожидаемо:** на какой-то итерации логи «застынут», `join()` не вернётся — это и есть дедлок. Остановите процесс вручную (или запускайте этот блок отдельно).

B) Ordered (фикс №1)

1. Замените процессор на `OrderedLockableResourceProcessor` (с `synchronized` внутри).
2. Тот же цикл `iterations = 1000`.
3. Ожидаем: все итерации завершаются; в логах видно «захват A → B» (или «B → A») **в одном порядке**.

C) TryLock (фикс №2)

1. Замените процессор на `TryLockLockableResourceProcessor` (`ReentrantLock` + `tryLock(timeout)` + `backoff`).
2. Тот же цикл `iterations = 1000`.
3. Ожидаем: дедлоков нет; видны «неудачная попытка второго лока → release первого → backoff → повтор → успех».

✓ Проверь себя

- На **Wrong** логи «замерли», `join()` не вернулся — дедлок показан.

- На **Ordered** — все 1000 итераций завершаются; порядок захвата в логах един.
- На **TryLock** — в логах видны ретраи/откаты; все итерации завершаются.
- Везде локи/мониторы освобождаются через `finally`.

Мини-чеклист (Deadlock)

- **Java 21**, запуск из `main`, параметры — константами в коде.
- Логи печатают **время** и **имя потока** на каждом важном событии.
- В Wrong-сценарии добавлена пауза между первым и вторым захватом.
- В TryLock-сценарии есть `tryLock(timeout)` и **backoff** 10–30 мс.
- Для Ordered захват всегда по возрастанию `id`.

Подсказки

- Дедлок = **несогласованный порядок захвата**. Простейший антидот — **тотальный порядок**.
- `tryLock(timeout) + backoff` — «вежливый» способ избежать взаимной блокировки без глобального порядка.
- Пишите логи в точках: «начало итерации», «вход в критсекцию», «взял/не взял лок», «backoff X мс», «выход», «конец итерации».

Утилита логирования (используйте везде, где есть критические события)

▼ утилита

```

1 /**
2  * Компактный logger: печатает время и имя потока в каждой строке
3  */
4 public final class Logger {
5     private Logger() {}
6     public static void log(String fmt, Object... args) {
7         String message = (args == null || args.length == 0) ? fmt :
String.format(fmt, args);
8         System.out.printf("[%tT.%1$tL][thread:%s] %s%n",
9             System.currentTimeMillis(),
10            Thread.currentThread().getName(),
11            message);
12     }

```

